

Claude Code: Getting Started Guide

For Mac Users with Local Files

What Is Claude Code?

Claude Code is a command-line tool that lets you work with Claude directly inside your computer's terminal, focused on files and projects on your machine. Unlike the `claude.ai` chat interface you're used to, Claude Code can read, write, and edit your actual files -- it's Claude working alongside your documents and folders, not just answering questions.

Think of the difference this way: `claude.ai` is a conversation. Claude Code is a collaborator who can open your files, make changes, and show you exactly what it did.

Before You Start: Two Things to Know

What is Terminal? Terminal is a text-based way to talk to your Mac. Instead of clicking icons, you type commands. It sounds intimidating but you only need about four commands to get going. It's located in Applications > Utilities > Terminal, or search for it with Spotlight (Cmd+Space, type "Terminal").

What subscription do you need? Claude Code works with a Claude Pro plan (\$20/month) or Claude Max plan (\$100/month). It uses the same account as `claude.ai`. If you're already on Pro, you're set.

Part 1: Installation

Step 1: Open Terminal

Press `Cmd + Space`, type `Terminal`, hit Enter. A window with a blinking cursor appears. That's it. You're in.

Step 2: Install Claude Code

Copy this entire line and paste it into Terminal, then press Enter:

```
curl -fsSL https://claude.ai/install.sh | bash
```

This downloads and installs Claude Code automatically. It takes about a minute. You'll see text scrolling -- that's normal.

Step 3: Verify It Worked

Type this and press Enter:

```
claude --version
```

You should see a version number printed back. If you see "command not found," something went wrong -- jump to the Troubleshooting section at the end.

Step 4: Run the Health Check

Type this and press Enter:

```
claude doctor
```

This checks your installation and tells you if anything needs attention. Green checkmarks mean you're good.

Part 2: First Login

Step 5: Authenticate

Type this and press Enter:

```
claude
```

The first time you run this, Claude Code will open a browser window and ask you to log in with your Claude.ai account. Log in the same way you do at claude.ai. Once you're authenticated, it saves your credentials -- you won't need to log in again.

After logging in, you'll see the Claude Code welcome screen. Type `/exit` or press `Ctrl+C` to leave for now.

Part 3: Understanding the Basics

How Claude Code Works

Every time you use Claude Code, you "point" it at a folder on your computer. Claude can see all the files inside that folder and will work within it. This is called your **working directory**.

The basic workflow is:

1. Navigate to a folder in Terminal
2. Type `claude` to start a session
3. Ask Claude to do something in plain English
4. Claude shows you what it plans to do and asks permission

5. You approve, and it makes the change

Claude never modifies your files without asking you first.

The Only Navigation Command You Need

To move to a folder in Terminal, type `cd` followed by the path. The easiest method: type `cd` (with a space after it), then drag a folder from Finder directly into the Terminal window. The path fills in automatically. Press Enter.

Example:

```
cd /Users/adamconner/Documents/my-project
```

Part 4: Setting Up Your First Project Folder

Step 6: Create a Test Folder

In Finder, create a new folder somewhere easy to find -- your Desktop or Documents folder works fine. Name it something like `claude-test`.

Step 7: Navigate to It

In Terminal:

```
cd ~/Desktop/claude-test
```

(If you put it in Documents, use `~/Documents/claude-test` instead.)

Step 8: Start Claude Code

```
claude
```

You'll see the welcome screen. You're now working inside your test folder.

Part 5: Essential Commands

Once Claude Code is running, you talk to it by typing. Here are the commands worth knowing:

What you type	What it does
Any plain English	Ask Claude to do something
<code>/help</code>	See all available commands
<code>/clear</code>	Clear the conversation and start fresh
<code>/exit</code>	Close Claude Code
Ctrl+C	Force quit if something gets stuck
<code>?</code>	See keyboard shortcuts

Slash commands (starting with `/`) are system commands for Claude Code itself. Everything else is a message to Claude.

Part 6: Four Starter Projects

These projects let you explore what Claude Code can do without needing any coding background. Each one teaches a different skill.

Project A: Organize a Messy Folder

What you'll learn: Claude Code reading your files and making suggestions.

Setup: Create a folder called `messy-docs`. Drop in 10-15 random files -- mix of PDFs, Word docs, text files, whatever. Name them messily (things like `doc1.docx`, `final_FINAL.pdf`, `notes copy.txt`).

Start Claude Code:

```
cd ~/Desktop/messy-docs
claude
```

Try these prompts:

What files are in this folder?

Suggest a folder structure that would organize these files logically.

Create subfolders based on your suggestion, but don't move any files yet.

Now move the files into the appropriate subfolders.

What to notice: Claude asks permission before moving anything. It tells you what it's doing at each step. You can say "no" or "wait" at any point.

Project B: Summarize and Analyze Documents

What you'll learn: Claude reading your actual files and extracting information.

Setup: Create a folder called `policy-docs`. Put in 3-5 PDFs or text files -- CAP reports, news articles you've saved, anything work-related works great here.

Start Claude Code:

```
cd ~/Desktop/policy-docs
claude
```

Try these prompts:

What documents do I have here?

Summarize each document in two sentences.

What are the common themes across all these documents?

Create a new file called `summary.md` with a one-page briefing covering the main points from all these documents.

What to notice: Claude reads the actual content of your files, not just the filenames. The output file it creates (`summary.md`) is a real file you can open in any text editor.

Project C: Build a Simple To-Do List App

What you'll learn: Claude creating files from scratch and doing iterative work.

Setup: Create an empty folder called `todo-app`.

Start Claude Code:

```
cd ~/Desktop/todo-app  
claude
```

Try these prompts:

Create a simple to-do list that I can use in my browser. It should let me add tasks, check them off, and delete them.

Claude will create an HTML file. Then try:

Open the file in my browser.

Add a feature that lets me set a priority (high, medium, low) for each task.

Change the color scheme to something more professional -- blues and grays.

What to notice: You're iterating. Each prompt builds on the last. Claude shows you the change before making it. The resulting file is real -- you can keep using it.

Project D: Draft and Refine a Document

What you'll learn: Using Claude Code for writing work, including edits and revisions.

Setup: Create a folder called `writing-project`. This works well with a real work task -- a memo, a policy brief outline, talking points, anything.

Start Claude Code:

```
cd ~/Desktop/writing-project  
claude
```

Try these prompts:

Create a file called `briefing.md`. Write a one-page briefing on the current state of federal AI regulation -- focus on the executive branch, key agencies, and the gap between existing statutory authority and AI oversight needs.

Then iterate:

The section on agency authority is too vague. Make it more specific -- name the agencies and what specific authorities they're using or could use.

Add a section at the end on what Congress has and hasn't done.

Make the whole thing more concise -- cut it by about 30%.

What to notice: This is closer to how you'd actually use Claude Code for policy work. The file persists on your computer. You can open it, edit it in any text editor, and come back to Claude Code to continue.

Part 7: Useful Patterns to Know

Plan Mode: Before Claude makes changes, you can ask it to plan first:

Press `(Shift + Tab)` to toggle Plan Mode. In Plan Mode, Claude describes what it will do without actually doing it. Good for bigger tasks where you want to review the approach first.

Continuing a session: If you close Terminal and come back later:

```
claude --continue
```

This picks up your most recent conversation.

Asking about Claude Code itself:

```
What can you do?
```

```
How do I give you a file that's outside this folder?
```

Claude Code can answer questions about its own capabilities.

Troubleshooting

"command not found" after installing: Close Terminal completely, reopen it, and try `claude --version` again. If it still fails, run the install command again.

Claude isn't reading my PDF: Try converting it to a text file first, or ask Claude: "Can you read PDF files?" -- it will tell you what it can and can't access.

Claude is making too many changes at once: Be more specific with smaller requests. Instead of "clean up this whole project," try "look at just this one file and tell me what you'd change."

Something went totally wrong: Type `/clear` to reset the conversation, or `/exit` and start fresh. Your files are only modified when you approve changes, so nothing should be permanently broken.

Quick Reference Card

Open Terminal: `Cmd+Space`, type "Terminal"

Navigate to folder: `cd ~/Desktop/folder-name`

Start Claude Code: `claude`

Get help: `/help`

Clear conversation: `/clear`

Exit: `/exit` or `Ctrl+C`

Health check: `claude doctor`

Continue last session: `claude --continue`

What's Next

Once you're comfortable with the basics, explore:

- **MCP servers** -- connectors that let Claude Code access tools like your calendar, Obsidian vault, or web search
- **CLAUDE.md files** -- a special file you can put in any project folder to give Claude persistent context about that project
- **Claude Code on the web** -- run longer tasks asynchronously at claude.ai/code

Official docs: <https://code.claude.com/docs/en/quickstart>